



Operating System

Lab 14

Fasiha Tariq – 53289

Submitted to:

Ma'am Ayesha Ikram

Date: November 10th, 2025

TASKS

1.

```
student@student-virtual-machine:~$ pico Thread1.c
student@student-virtual-machine:~$ cat Thread1.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>    // Header file for sleep(). man 3 sleep for details.
#include <pthread.h>

// A normal C function that is executed as a thread
// when its name is specified in pthread_create()
void *myThreadFun(void *vargp)
{
    sleep(1);
    printf("Printing GeeksQuiz from Thread \n");
    return NULL;
}

int main()
{
    pthread_t thread_id;
    printf("Before Thread\n");
    pthread_create(&thread_id, NULL, myThreadFun, NULL);
    pthread_join(thread_id, NULL);
    printf("After Thread\n");
    exit(0);
}
student@student-virtual-machine:~$ gcc Thread1.c -o Thread1
student@student-virtual-machine:~$ ./Thread1
Before Thread
Printing GeeksQuiz from Thread
After Thread
```

2.

```
student@student-virtual-machine:~$ pico Thread2.c
student@student-virtual-machine:~$ cat Thread2.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <pthread.h>

// Let us create a global variable to change it in threads
int g = 0;

// The function to be executed by all threads
void *myThreadFun(void *vargp)
{
    // Store the value argument passed to this thread
    int *myid = (int *)vargp;

    // Let us create a static variable to observe its changes
    static int s = 0;

    // Change static and global variables
    ++s; ++g;

    // Print the argument, static and global variables
    printf("Thread ID: %d, Static: %d, Global: %d\n", *myid, ++s, ++g);
    return NULL;
}

int main()
{
    int i;
    pthread_t tid;

    // Let us create three threads
    for (i = 0; i < 3; i++)
        pthread_create(&tid, NULL, myThreadFun, (void *)&i);

    pthread_exit(NULL);
    return 0;
}
student@student-virtual-machine:~$ gcc Thread2.c -o Thread2
student@student-virtual-machine:~$ ./Thread2
Thread ID: -106961344, Static: 2, Global: 2
Thread ID: -251660736, Static: 4, Global: 4
Thread ID: -251660736, Static: 6, Global: 6
```

Explanation:

- Global variable g and Static variable s is incremented twice in each thread
- Static variable s starts at 0.
- Both variables are shared, so their values continue increasing across threads.

3.

```
student@student-virtual-machine:~$ cat Thread3.c
#include <stdio.h>
#include <pthread.h>
#include <stdlib.h>

// Thread function
void* printMessage(void* arg) {
    int threadNum = *((int*)arg);
    printf("Hello World! This is thread %d\n", threadNum);

    // pthread_exit(NULL);
}

int main() {
    pthread_t threads[3];
    int threadArgs[3];
    int i;

    for (i = 0; i < 3; i++) {
        threadArgs[i] = i + 1;
        pthread_create(&threads[i], NULL, printMessage, (void*)&threadArgs[i]);
    }

    for (i = 0; i < 3; i++) {
        pthread_join(threads[i], NULL);
    }

    printf("All threads completed.\n");
    return 0;
}
student@student-virtual-machine:~$ gcc Thread3.c -o Thread3
student@student-virtual-machine:~$ ./Thread3
Hello World! This is thread 2
Hello World! This is thread 3
Hello World! This is thread 1
All threads completed.
```

With Exit:

```
student@student-virtual-machine:~$ pico Thread3.c
student@student-virtual-machine:~$ gcc Thread3.c -o Thread3
student@student-virtual-machine:~$ ./Thread3
Hello World! This is thread 1
Hello World! This is thread 2
Hello World! This is thread 3
All threads completed.
```

4.

```
student@student-virtual-machine:~$ pico Thread4.c
student@student-virtual-machine:~$ cat Thread4.c
#include <stdio.h>
#include <pthread.h>
#include <stdlib.h>

// Function executed by each thread
void* threadFunction(void* arg) {
    int threadNum = *((int*)arg);
    printf("Hello from thread %d\n", threadNum);
    pthread_exit(NULL);
}

int main() {
    pthread_t threads[3];
    int threadNums[3];
    int i;

    for (i = 0; i < 3; i++) {
        threadNums[i] = i + 1;
        pthread_create(&threads[i], NULL, threadFunction, &threadNums[i]);
    }

    for (i = 0; i < 3; i++) {
        pthread_join(threads[i], NULL);
    }

    printf("Main thread finished.\n");
    return 0;
}
student@student-virtual-machine:~$ gcc Thread4.c -o Thread4
student@student-virtual-machine:~$ ./Thread4
Hello from thread 1
Hello from thread 2
Hello from thread 3
Main thread finished.
```